

Token Lifecycle - Mamba

Qi Zao Lim

May 2026

0.1 The Mamba Token Lifecycle Framework

Why SSMs and Mamba While Transformers rely on attention mechanisms, SSMs (like Mamba) act like upgraded Recurrent Neural Networks. SSMs excel in linear inference speeds ($O(N)$ speed relative to transformer $O(N^2)$) and low memory usage, but lack the robust understanding of context that transformers possess, making it a poor option for complex reasoning models like LLMs. This creates a tradeoff between speed and accuracy for LLMs.

Mamba, as an evolution of SSMs, is the first alternative to the transformer architecture while having the speed of SSMs. It does so through two innovations: the first is the selective state space model, which provides Mamba with a crucial capability previously possessed only by transformer models: the ability to selectively focus on or ignore specific parts of past input history based on their present relevance. The other is hardware-aware parallel scan, an algorithm that optimizes the way a graphics processing unit (GPU) handles the model's computations in its memory hierarchy to maximize speed and computational efficiency.

The Mamba architecture fundamentally reimagines how tokens flow through a sequence. While Transformers rely on explicit, all-to-all geometric forces, Mamba operates as a continuous dynamical system optimized for hardware efficiency. However, beneath this rigid sequential pipeline lies a massive, distributed ensemble of hidden, dynamic token interactions.

0.1.1 Stage 1: Initialization & Conditioning (Encoding)

This stage sets the foundation, transforming discrete text into a mathematical object capable of being manipulated within a continuous state space.

Linear Projection & Branching After raw tokens are converted into continuous vector embeddings, the input vector undergoes a linear projection that expands its dimensionality. Crucially, Mamba splits this expanded token into two distinct parallel computational paths: one destined for the Selective State Space Model (SSM) core (x_{proj}), and the other for a parallel multiplicative gating mechanism (z_{proj}) (Gu & Dao, 2023).

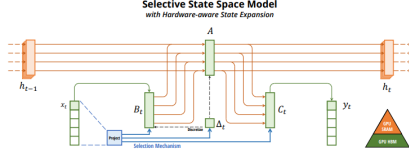


Figure 1: Selective SSM architecture

Inherent Temporal Grounding Unlike Transformers, Mamba does not require explicit positional encodings (like sinusoidal waves or RoPE) to be injected into the token. Because its foundation relies on continuous-time differential equations, the sequence and temporal order are inherently baked into the mathematics of the state space.

0.1.2 Stage 2: Sequence & Spatial Mixing (Interaction)

This stage dictates how a token gathers context from its environment. On the surface, Mamba appears to destroy standard token interaction, but mathematically, it recreates it efficiently through the State Space Model (SSM) recurrence.

$$h'(t) = Ah(t) + Bx(t) \quad (1)$$

To process discrete discrete tokens in a sequence, this continuous system is discretized using a timescale parameter Δ , transforming it into the foundational SSM recurrence: $h_t = \bar{A}h_{t-1} + \bar{B}x_t$

$y_t = Ch_t$ Here, x_t is the current token input, h_t is the hidden state (the accumulated context), and y_t is the output. The matrices $\bar{A}$ and $\bar{B}$ are the discretized versions of the continuous parameters A and B . Evidently, there is a radical mechanistic difference in how these architectures process tokens. Transformers use all-to-all dense interactions, while Mamba relies on sequential, state-mediated updates. Tokens are processed sequentially and squashed into a compressed 1D hidden state, seemingly destroying the direct token-to-token maps seen in Transformers.

Hidden Attention Generation Mamba does not merely approximate attention, but rather generate “hidden” attention matrices on the fly (Dao Gu, 2024). By unrolling this sequential process mathematically, it is proven that Mamba implicitly forms highly structured dependencies over time, dynamically prioritizing both recent and highly salient distant tokens without ever actually computing a full $\mathcal{O}(n^2)$ matrix [cite: 333].

0.1.3 Stage 3: Transformation & Conditional Routing (Processing)

Once context is flowing into the hidden state, this stage covers the non-linear transformations and the model’s core decision-making process.

Local Convolutional Extraction Before entering the SSM, the token representation is fed through a 1D convolution layer (Gu & Dao, 2023). This acts as a localized filter, extracting immediate syntactic patterns between neighboring tokens, which frees the deeper state space to focus entirely on global dependencies. The output of the convolution layer serves as the input to a nonlinear activation function. For instance, the original Mamba infrastructure uses Sigmoid Linear Unit (SiLU). The separate gating mechanism path, z_{proj} also goes through a non linear activation function.

Input-Dependent Selectivity The defining feature of Mamba is its input-dependent gating—unlike older fixed-state models, Mamba decides what to remember based on what it is currently reading[cite: 302]. The mathematical parameters defining the state space transition are continuously recalculated based on the specific token at that exact step.

0.1.4 Stage 4: Depth & Temporal Evolution (Integration)

This stage models the cumulative trajectory of the token as it updates and stabilizes across network depth. Standard recurrent networks and basic SSMs suffer from exponential memory decay, where old tokens simply fade away. By dynamically adjusting its mathematical projections on a per-token basis, Mamba actively counteracts that natural decay.

0.1.5 Stage 4: Depth & Temporal Evolution (Integration)

This stage models the cumulative trajectory of the token as it updates and stabilizes across network depth. This is where Mamba’s “Selective” mechanism mathematically grounds the model’s ability to retain context and filter noise.

Input-Dependent Selectivity & Counteracting Decay Standard recurrent networks and vanilla SSMs suffer from exponential memory decay because their transition matrices (A , B , C , and Δ) are static—they apply the exact same mathematical operation regardless of the input.

Mamba breaks this limitation and achieves “selectivity” by making the step size (Δ), B , and C explicitly dependent on the current token input x_t :

$$B_t = \text{Linear}(x_t)$$

$$C_t = \text{Linear}(x_t)$$

$$\Delta_t = \text{Softplus}(\text{Parameter} + \text{Linear}(x_t))$$

Because $\bar{A}$ and $\bar{B}$ are derived using the now-dynamic Δ_t , the entire recurrence equation becomes fundamentally input-dependent. By dynamically adjusting these parameters on a per-token basis, Mamba actively counteracts that natural exponential decay. If a token is irrelevant, the model can mathematically “shrink” Δ_t to ignore it; if a token is critical, it can expand it to heavily update the hidden state.

Multi-Scale Filtering Because the math is dynamic, the token acts as a sophisticated, multi-scale filter. It deliberately holds onto specific, highly relevant historical tokens for massive context windows, while aggressively forgetting useless intermediary tokens.

Circuit Complexity Equivalence Despite processing tokens one by one through a constrained state space, Mamba suffers no fundamental theoretical disadvantage in expressivity; it is mathematically capable of computing the exact same class of functions over a token sequence as a full self-attention mechanism.

0.1.6 Stage 5: Projection & Emission (Output)

The final stage ends the token’s internal journey, aggressively shifting from continuous latent states back out to discrete predictive probabilities.

Gated Recombination The output emerging from the sequential SSM path is multiplied element-wise by the activation from the parallel gating path established back in Stage 1. This gate acts as a volume dial—suppressing the contextual update if irrelevant, or amplifying it if highly semantic (Gu & Dao, 2023).

Residual Accumulation The resulting recombined vector is projected back down to its original dimension and added directly to the residual stream.

Emission The final token representation undergoes normalization and a linear projection out to the vocabulary size, generating a probability distribution to autoregressively predict the next word.